

SOFTWARE SUPPLY CHAIN RISK ANALYZER (SBOM ANALYZER)

Hackathon Submission

Problem Statement: Software Supply Chain Risk Scorer (SBOM Analyzer)

Submitted By

Team Name: HackHer

Team Members: Vinothini G

Shazmina A J

College: PSG College Of Technology

Department: Information Technology

Date: 11-07-2026

Abstract

Modern enterprise applications rely heavily on third-party open-source libraries. While these libraries accelerate software development, they also introduce security vulnerabilities, outdated dependencies, and software license compliance issues. Organizations often lack visibility into the software components used across multiple applications, making it difficult to respond quickly when new vulnerabilities are disclosed.

This project presents a Software Supply Chain Risk Analyzer, an automated SBOM (Software Bill of Materials) analysis platform that detects vulnerable libraries, identifies outdated dependencies, checks license compatibility, computes an overall software risk score, and provides security recommendations through an interactive web dashboard.

The proposed solution processes SBOM data stored in CSV and JSON files, cross-references them against a vulnerability database and license rules, and generates a comprehensive security report to assist developers and security analysts in prioritizing remediation efforts.

Problem Statement

Organizations maintain numerous enterprise applications that depend on open-source software libraries. Each application may pull in dozens or even hundreds of transitive dependencies, and this dependency tree changes constantly as libraries are updated, replaced, or deprecated. Without a systematic way to track these components, security teams are effectively blind to a large portion of their software attack surface.

This lack of visibility becomes especially dangerous when a critical vulnerability is publicly disclosed. Security teams must quickly determine which applications, if any, use the affected library and version — a process that today is largely manual, slow, and error-prone.

Challenges Include

- Lack of visibility into software dependencies — teams do not have a single source of truth listing every library used across applications.
- Vulnerable third-party libraries — open-source components may contain known Common Vulnerabilities and Exposures (CVEs) that go unnoticed.
- Outdated software components — libraries that have not been upgraded in a long time accumulate unpatched security and stability issues.
- License compatibility violations — incompatible open-source licenses (e.g., GPL used inside a proprietary product) can create legal risk.
- Manual dependency tracking — spreadsheets and tribal knowledge do not scale across dozens of applications and teams.
- Slow incident response — when a new CVE is announced, it can take days or weeks to determine organizational exposure.

Impact on Organizations

- Increased risk of data breaches and security incidents caused by unpatched, known vulnerabilities.
- Regulatory and compliance exposure from unmanaged open-source license obligations.
- Wasted engineering time on manual audits that could be automated.
- Reputational damage following publicly disclosed supply-chain attacks.

The goal of this project is to automate dependency analysis and generate a clear, actionable software supply chain risk report that security teams and developers can act on immediately.

Objectives

The project is guided by a set of clear, measurable objectives that together define the scope and success criteria of the platform. These objectives were chosen to directly address the gaps identified in the problem statement.

1. Parse SBOM files automatically — accept CSV-based Software Bill of Materials uploads and extract structured dependency data without manual data entry.
2. Detect vulnerable libraries — cross-reference each dependency's name and version against a known vulnerability database to flag CVEs.
3. Identify outdated libraries — compare the installed version of each library against the latest available version to highlight technical debt.
4. Detect license conflicts — check each library's license against a configurable set of organizational license policies.
5. Compute overall software risk score — combine vulnerability severity, outdated-ness, and license risk into a single quantifiable score per application.
6. Generate an interactive dashboard — present findings through charts, tables, and summaries that are easy to interpret at a glance.
7. Provide security recommendations — translate raw findings into prioritized, actionable remediation steps for developers.
8. Improve software supply chain visibility — give security and engineering leadership a consolidated view across all applications.
9. Reduce manual dependency analysis — eliminate the need for spreadsheet-based tracking and manual CVE lookups.

Success Criteria

- The system correctly parses SBOM CSV files without data loss or misclassification.
- Risk scores are consistent, reproducible, and easy to explain to non-technical stakeholders.
- The dashboard loads and renders results within a few seconds for typical SBOM file sizes.
- Recommendations are specific enough that a developer can act on them without further research.

Existing System

Current organizations typically rely on manual, spreadsheet-driven processes to track their open-source software usage. These processes were adequate when applications had a handful of dependencies, but modern applications routinely depend on hundreds of packages pulled in transitively, making manual tracking impractical.

Current organizations typically use:

- Excel spreadsheets — maintained by individual developers or teams, quickly falling out of date.
- Manual dependency tracking — engineers manually note down library names and versions during development.
- Manual vulnerability lookup — security analysts search CVE databases and vendor advisories one library at a time.
- Manual license verification — legal or compliance teams review license files individually before release.

Disadvantages

- Time consuming — manually correlating hundreds of dependencies against vulnerability databases can take days.
- Human errors — manual data entry and cross-referencing are prone to typos and missed entries.
- No centralized visibility — each team keeps its own tracking sheet, with no organization-wide view.
- Slow vulnerability response — by the time a vulnerable library is identified, attackers may already have exploited it.
- Difficult to analyze multiple applications — comparing risk across dozens of applications is impractical by hand.

Existing vs. Proposed Approach

Existing (Manual) Approach	Proposed (Automated) Approach
Spreadsheet-based tracking	Structured SBOM ingestion (CSV/JSON)
Manual CVE lookup per library	Automated cross-referencing against a vulnerability database
No unified risk score	Computed, consistent overall risk score
Static, outdated reports	Interactive, always-current dashboard
Hours to days per application	Seconds to minutes for an entire portfolio

Proposed System

The proposed platform automates the complete dependency analysis process, replacing manual spreadsheet tracking with a structured, repeatable pipeline. A user simply uploads a CSV-based SBOM, and the system takes care of parsing, cross-referencing, scoring, and visualizing the results — turning what used to take days of manual effort into a task that completes in seconds.

Features

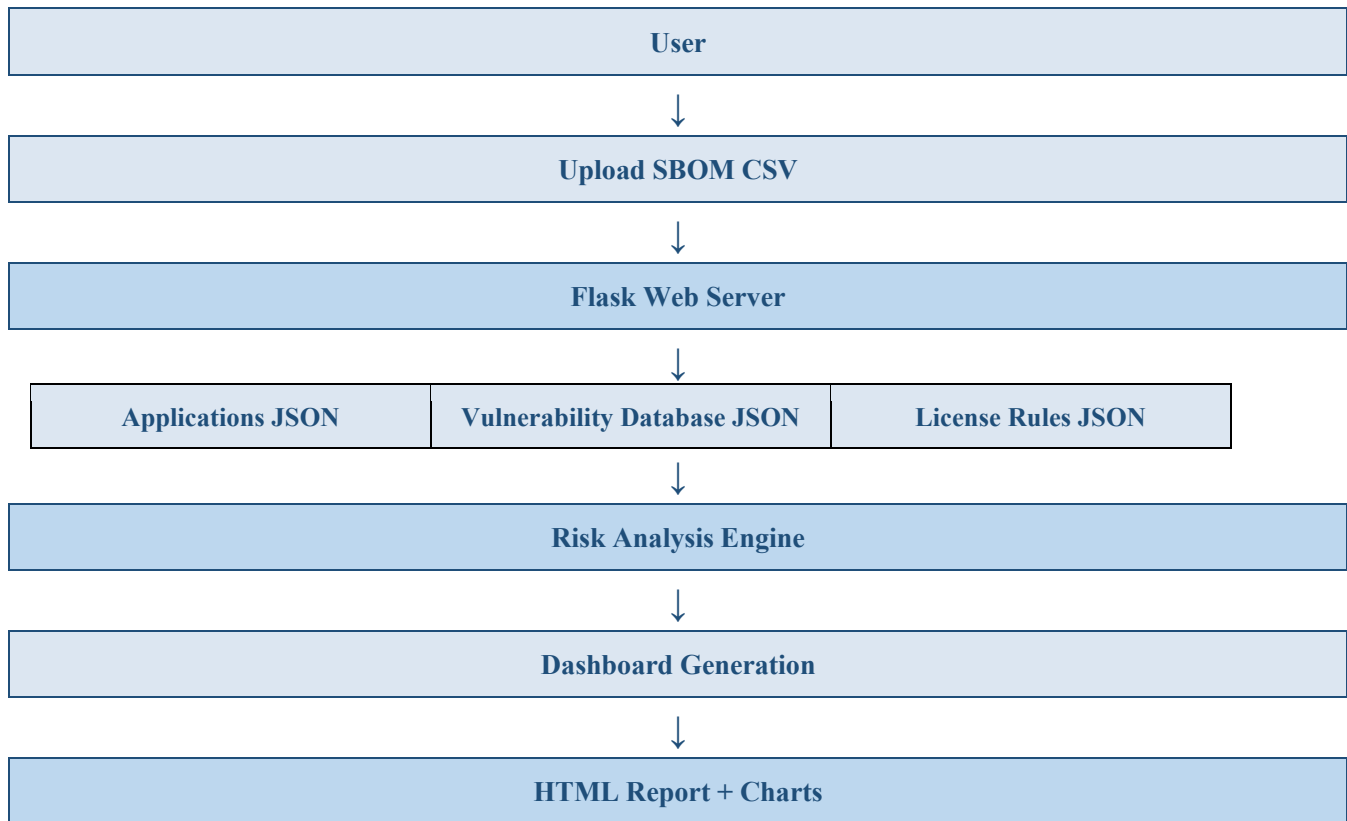
- Upload SBOM files — a simple web-based upload form accepts SBOM data in CSV format.
- Parse CSV data — the backend reads and validates the uploaded file, extracting library names, versions, and associated applications.
- Read JSON databases — application metadata, vulnerability records, and license rules are stored as structured JSON for fast lookup.
- Detect vulnerabilities — each dependency is checked against known CVEs and assigned a severity level.
- License compliance analysis — detected licenses are checked against configurable organizational policy rules.
- Outdated dependency detection — installed versions are compared to the latest known release to flag technical debt.
- Risk score calculation — vulnerability, license, and outdated-ness signals are combined into one overall risk score.
- Dashboard visualization — results are rendered as pie charts, bar charts, and sortable tables using Chart.js.
- Security recommendations — the system generates prioritized, human-readable remediation guidance.

Advantages

- Fast — analysis that previously took hours now completes in seconds.
- Accurate — automated cross-referencing removes human transcription errors.
- Automated — no manual spreadsheet maintenance is required.
- User friendly — a clean web dashboard makes findings accessible to both developers and non-technical stakeholders.
- Scalable — the same pipeline can analyze one application or an entire portfolio with minimal additional effort.

System Architecture

The architecture follows a straightforward, layered flow: a user uploads an SBOM file through the web interface, the Flask backend parses and enriches that data using three supporting JSON sources, the risk analysis engine computes findings and scores, and the results are rendered back to the user as an interactive dashboard.



Component Descriptions

- User — the developer or security analyst who initiates the analysis by uploading an SBOM file.
- Upload SBOM CSV — the file upload interface that accepts the Software Bill of Materials in CSV format.
- Flask Web Server — the Python backend that handles requests, orchestrates parsing, and serves the dashboard.
- Applications JSON — metadata describing each application, such as name, owner, and environment.
- Vulnerability Database JSON — a structured store of known CVEs mapped to library names and affected versions.
- License Rules JSON — organizational policy defining which open-source licenses are approved, restricted, or prohibited.
- Risk Analysis Engine — the core logic that cross-references dependencies against all three data sources and computes risk.
- Dashboard Generation — assembles the processed findings into charts, tables, and summary statistics.
- HTML Report + Charts — the final rendered output delivered back to the user's browser.

Algorithm

The core analysis algorithm executes in a linear sequence each time an SBOM file is uploaded. Each step builds on the output of the previous one, ending with a fully populated dashboard.

10. Read uploaded SBOM CSV — parse the file and extract each dependency's name, version, and associated application.
11. Load application metadata — retrieve contextual information about each application from applications.json.
12. Load vulnerability database — load the known CVE records from the vulnerability database JSON file.
13. Load license rules — load the organization's license policy definitions from the license rules JSON file.
14. Compare each dependency — for every parsed library, check its name and version against all three loaded data sources.
15. Detect vulnerabilities, license conflicts, and outdated libraries — flag any matches found during comparison.
16. Assign risk level — classify each dependency as Critical, High, Medium, Low, or Safe based on the findings.
17. Calculate overall risk score — aggregate individual risk levels into one overall score per application.
18. Display dashboard — render the final results as charts, tables, and recommendations for the user.

Risk Level Classification

Risk Level	Criteria
Critical	Known CVE with high/critical severity, actively exploited
High	Known CVE with medium severity, or severely outdated version
Medium	License conflict, or moderately outdated version
Low	Minor version lag with no known vulnerabilities
Safe	No vulnerabilities, license conflicts, or significant version lag

Technologies Used

The platform was built using a lightweight, widely-supported technology stack chosen for rapid development within the hackathon timeframe while remaining production-realistic. Python and Flask power the backend, JSON files serve as a simple embedded database, and Chart.js drives an interactive front-end dashboard.

Technology	Purpose	Notes
Python	Backend	Core language for parsing, analysis, and scoring logic
Flask	Web Framework	Lightweight framework serving routes and the dashboard
HTML	User Interface	Structure for the dashboard and upload pages
CSS	Styling	Visual styling and responsive layout of the dashboard
JavaScript	Charts	Client-side logic driving interactive chart rendering
Chart.js	Graphs	Library used to render pie and bar charts
JSON	Database	Stores application metadata, vulnerabilities, and license rules
CSV	SBOM Input	Input format for uploaded Software Bill of Materials files
VS Code	Development	Primary IDE used throughout development

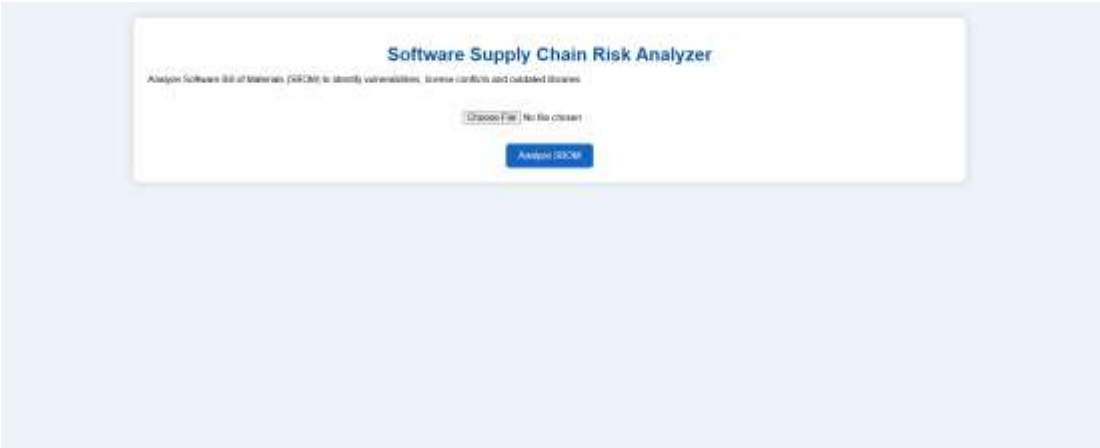
Why This Stack

- Python was chosen for its rich ecosystem of data-processing libraries and rapid prototyping speed.
- Flask keeps the backend minimal and easy to extend, avoiding unnecessary framework overhead.
- JSON files provide a zero-setup database suitable for a hackathon timeline, while mirroring the structure a real database would use.
- Chart.js was selected for its simplicity and strong out-of-the-box support for pie and bar charts.

User Interface

The following screens illustrate the platform's interactive dashboard. Insert screenshots from the running application in the placeholders below.

Dashboard



Overview dashboard summarizing total libraries scanned, overall risk score, and key metrics.

Pie Chart and Graph



Visual Comparison of risk levels across applications or dependency categories.

Risk Table

Application ID	Application Name	Library	Version	License	Dependency Type	Last Updated	Risk
APP-001	CustomerPortal	micrometer-core	3.0.10	Apache-2.0	direct	2025-01-30	Safe
APP-001	CustomerPortal	sqlalchemy	5.13.5	ISC	direct	2022-03-18	Outdated
APP-001	CustomerPortal	netty-all	2.7.4	Unlicense	direct	2025-05-01	Safe
APP-001	CustomerPortal	json-iterator	2.3.2	BSD-2-Clause	direct	2024-12-30	Safe
APP-001	CustomerPortal	log4j-api	5.1.3	Apache-2.0	direct	2025-07-11	Safe
APP-001	CustomerPortal	commons-collections4	4.7.1	MIT	direct	2021-03-31	Outdated
APP-001	CustomerPortal	commons-codec	4.11.10	BSD-2-Clause	direct	2024-08-16	Safe
APP-001	CustomerPortal	gson	1.6.10	Dual-MIT/Commercial	direct	2024-10-04	Safe
APP-001	CustomerPortal	lombok	4.1.2	BSD-3-Clause	direct	2023-10-31	Safe
APP-001	CustomerPortal	workzeug	1.7.10	MIT	direct	2022-02-23	Outdated
APP-001	CustomerPortal	jackson-annotations	5.0.6	BSD-3-Clause	direct	2026-01-01	Safe
APP-001	CustomerPortal	resilience4j	1.8.0	ISC	direct	2024-11-11	Safe
APP-001	CustomerPortal	stfl4j-api	2.8.8	MPL-2.0	direct	2025-07-03	Safe
APP-001	CustomerPortal	go-jwt	5.10.5	UNKNOWN	direct	2026-01-03	License Conflict
APP-001	CustomerPortal	spring-security-core	1.2.0	MIT	direct	2026-02-19	Safe
APP-001	CustomerPortal	snakeyaml	3.1.0	MIT	direct	2025-06-09	Safe
APP-001	CustomerPortal	opentelemetry-api	4.10.2	ISC	direct	2025-01-15	Safe
APP-001	CustomerPortal	logback-core	3.4.2	BSD-3-Clause	direct	2026-01-03	Safe
APP-001	CustomerPortal	pyyaml	5.7.10	Unlicense	direct	2021-12-11	Outdated
APP-001	CustomerPortal	jackson-databind	1.1.6	BSD-3-Clause	direct	2025-06-23	Safe

Detailed tabular listing of each dependency with its associated risk findings.

Recommendation Section

Security Recommendations

- Update vulnerable libraries to the latest secure version.
- Replace incompatible licenses with MIT or Apache licensed libraries.
- Remove outdated dependencies that have not been maintained.
- Apply vendor patches for affected components.
- Review transitive dependencies regularly.

Analyze Another File

Actionable security recommendations generated based on the analysis results.

Results

Testing the platform against sample SBOM datasets confirmed that it reliably classifies dependencies and produces a clear, consolidated risk picture. The system successfully identifies:

- Critical libraries — dependencies with actively exploited or high-severity known vulnerabilities.
- Outdated libraries — dependencies lagging significantly behind their latest stable release.
- License conflicts — dependencies whose license terms violate organizational policy.
- Safe libraries — dependencies with no detected vulnerabilities, license issues, or major version lag.

Sample Output Summary

The Dashboard Displays

- Total Libraries — the full count of dependencies scanned across the uploaded SBOM.
- Risk Score — a single aggregated score summarizing the overall health of the application's dependencies.
- Pie Chart — proportion of libraries in each risk category.
- Bar Chart — risk comparison across applications or dependency categories.
- Security Recommendations — prioritized, actionable remediation guidance.

Benefits

- Faster vulnerability analysis — reduces a multi-day manual audit to a near-instant automated scan.
- Better software visibility — gives teams a single, centralized view of dependency risk.
- Automated reporting — eliminates manual spreadsheet maintenance and repetitive CVE lookups.

Future Enhancements

While the current prototype demonstrates the core value of automated SBOM risk analysis, several enhancements would extend it into a production-grade platform suitable for continuous use across a large organization.

- CVE API Integration — replace the static local vulnerability JSON with live calls to public CVE data feeds.
- NVD Database — integrate directly with the National Vulnerability Database for authoritative, up-to-date CVE records.
- Real-time vulnerability updates — automatically re-scan applications whenever new vulnerabilities are disclosed.
- PDF report generation — allow users to export dashboard findings as shareable, downloadable PDF reports.
- User authentication — add login and role-based access control to protect sensitive dependency data.
- AI-based remediation suggestions — use machine learning to recommend safe upgrade paths automatically.
- Dependency graph visualization — display transitive dependency relationships as an interactive graph.
- Cloud deployment — host the platform on a cloud provider for organization-wide, always-on access.
- Email alerts — notify relevant teams automatically when a new critical vulnerability affects their application.
- GitHub integration — automatically pull SBOM data directly from connected GitHub repositories.

Conclusion

The Software Supply Chain Risk Analyzer provides an efficient solution for analyzing software dependencies and identifying security risks. It automates vulnerability detection, license compliance checking, outdated dependency analysis, and risk score computation through an intuitive web dashboard, replacing what has traditionally been a slow, manual, and error-prone process.

By consolidating dependency data from CSV-based SBOM files with structured vulnerability and license information, the platform gives development and security teams a single, trustworthy source of truth for supply chain risk. This directly addresses the visibility gap that leaves many organizations exposed to known, preventable vulnerabilities.

The proposed solution reduces manual effort, improves software visibility, and enables organizations to respond rapidly to newly discovered software vulnerabilities. With the future enhancements outlined in this document — particularly live CVE integration and cloud deployment — the platform has a clear path toward becoming a production-ready tool for continuous software supply chain risk management.

References

The following resources informed the design, security practices, and technical implementation of this project.

- OWASP — Open Worldwide Application Security Project, referenced for supply chain and dependency security best practices.
- NIST Cybersecurity Framework — used as a guiding framework for risk classification and organizational security posture.
- CVE Database — the industry-standard catalog of publicly disclosed cybersecurity vulnerabilities.
- Flask Documentation — official documentation used for backend routing and server implementation.
- Chart.js Documentation — official documentation used to implement dashboard pie and bar charts.
- Python Documentation — official language reference used throughout backend development.

Acknowledgement

We sincerely thank our faculty members, mentors, and the hackathon organizers for providing us the opportunity to develop this project. Their guidance and support helped us successfully design and implement the Software Supply Chain Risk Analyzer.

We are also grateful to the broader open-source community, whose tools, documentation, and shared knowledge made it possible to build this platform within the constraints of a hackathon timeline. Finally, we thank our teammates for their collaboration, effort, and dedication throughout the development process.